

Optimization-based machine learning for computational imaging

Neural networks

Nelly Pustelnik

CNRS, Laboratoire de physique, ENSL, F-69342 Lyon, France



FONDATION
SIMONE ET CINO
DEL DUCA
INSTITUT DE FRANCE

Motivation

Inference framework: feed-forward NN

$$\begin{aligned}(\forall \mathbf{x}^{[0]} \in \mathbb{R}^{N_0}) \quad \mathbf{x}^{[K]} &= \mathbf{f}_{\Theta}^K(\mathbf{x}^{[0]}) \\ &= \mathfrak{T}_{\Theta_K} \circ \dots \circ \mathfrak{T}_{\Theta_1}(\mathbf{x}^{[0]}),\end{aligned}$$

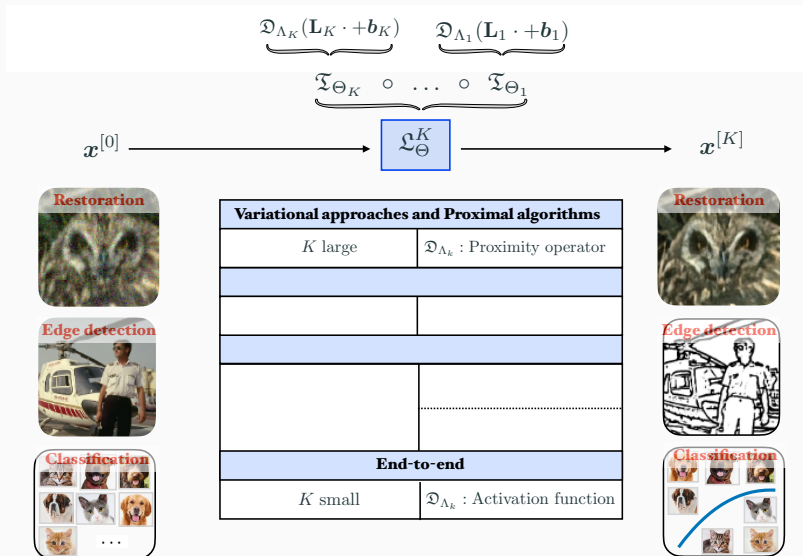
Layer/iteration

$$\mathfrak{T}_{\Theta_k} : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k} : \mathbf{x} \mapsto \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k),$$

- $\mathbf{L}_k : \mathbb{R}^{N_{k-1}} \rightarrow \mathbb{R}^{N_k}$: linear operator,
- $\mathbf{b}_k \in \mathbb{R}^{N_k}$: shift parameter,
- $\mathfrak{D}_{\Lambda_k} : \mathbb{R}^{N_k} \rightarrow \mathbb{R}^{N_k}$: nonlinear operator parametrized by Λ_k .

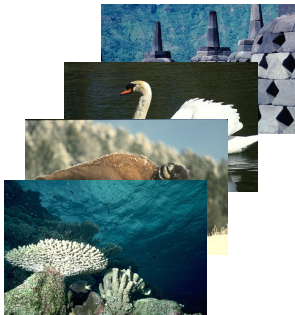
Parameters: $\Theta = \cup_{k=1}^K \Theta_k$ with $\Theta_k = \{\Lambda_k, \mathbf{L}_k, \mathbf{b}_k\}$.

Full picture



Training

- **Dataset** : $\mathcal{S} = \{(\bar{x}_\ell, z_\ell) \in \mathbb{R}^N \times \mathbb{R}^M \mid \ell \in \mathbb{I} \cup \mathbb{J}\}$



$\bar{x}_\ell$



z_ℓ

- **Dataset** : $\mathcal{S} = \{(\bar{x}_\ell, z_\ell) \in \mathbb{R}^N \times \mathbb{R}^M \mid \ell \in \mathbb{I} \cup \mathbb{J}\}$
 - *training set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{I}}$ with size $|\mathbb{I}|$
 - *testing set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{J}}$ with size $|\mathbb{J}|$

Training

- **Dataset** : $\mathcal{S} = \{(\bar{x}_\ell, z_\ell) \in \mathbb{R}^N \times \mathbb{R}^M \mid \ell \in \mathbb{I} \cup \mathbb{J}\}$
 - *training set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{I}}$ with size $|\mathbb{I}|$
 - *testing set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{J}}$ with size $|\mathbb{J}|$
- **Training stage**: find $\hat{\Theta} \in \underset{\Theta}{\text{Argmin}} \frac{1}{|\mathbb{I}|} \sum_{j \in \mathbb{I}} \ell(\bar{x}_j, f_{\Theta}^K(z_j))$.
 - ℓ : loss function
 - High-dimensional non-convex optimization problem
 - Use of auto-differentiation in standard toolboxes (e.g., Pytorch).

- **Dataset** : $\mathcal{S} = \{(\bar{x}_\ell, z_\ell) \in \mathbb{R}^N \times \mathbb{R}^M \mid \ell \in \mathbb{I} \cup \mathbb{J}\}$
 - *training set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{I}}$ with size $|\mathbb{I}|$
 - *testing set* $(\bar{x}_\ell, z_\ell)_{\ell \in \mathbb{J}}$ with size $|\mathbb{J}|$
- **Training stage**: find $\hat{\Theta} \in \underset{\Theta}{\operatorname{Argmin}} \frac{1}{|\mathbb{I}|} \sum_{j \in \mathbb{I}} \ell(\bar{x}_j, f_{\Theta}^K(z_j))$.
 - ℓ : loss function
 - High-dimensional non-convex optimization problem
 - Use of auto-differentiation in standard toolboxes (e.g., Pytorch).
- **Evaluation**: A properly trained network must satisfy

$$(\forall j \in \mathbb{J}) \quad \bar{x}_j \approx f_{\Theta}^K(z_j)$$

NN example: Multi-Layer-Perceptron (MLP)

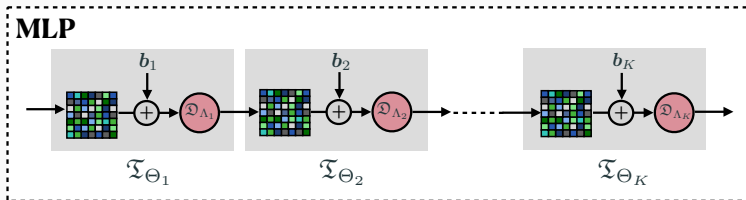
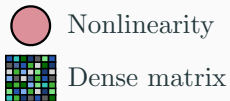
Definition: [Scaman, Virmaux, 2018]

A K -layer MLP is of the form of

$$f_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K} \quad \text{with} \quad \mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k),$$

where $(\mathbf{L}_k)_{1 \leq k \leq K}$ are dense matrices (i.e., fully connected layers).

Remark: When $K = 1$, the network is then referred to as a perceptron.



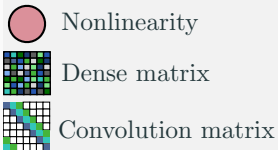
NN example: DnCNN

Definition: DnCNN architecture takes the form

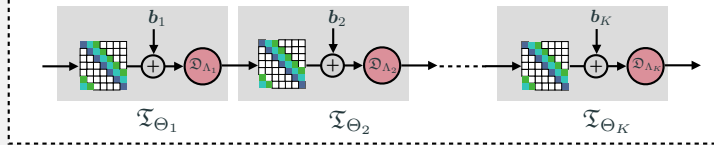
$$\mathbf{f}_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K} \quad \text{with} \quad \mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k)$$

- $\mathfrak{D}_{\Lambda_1}$: ReLU activation function
- $(\mathfrak{D}_{\Lambda_k})_{2 \leq k \leq K-1}$: compositions of batch normalization and ReLU
- $\mathfrak{D}_{\Lambda_K}$: identity
- $(\mathbf{L}_k)_{1 \leq k \leq K}$: convolution operators

Remark: Introduced as a denoising network [Zhang et al., 2017]



DnCNN



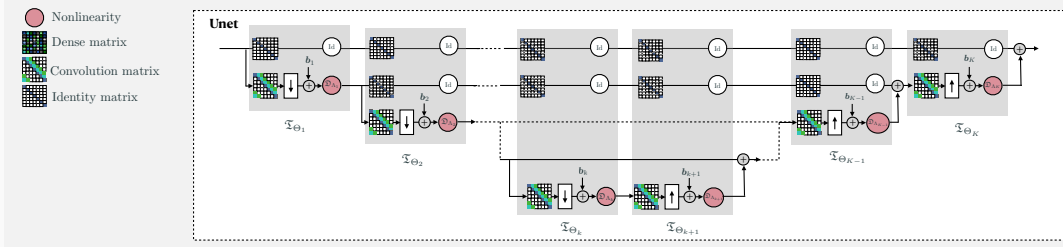
NN example: U-Net

Definition: Unet is an encoder-decoder architecture defined by

$$f_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K} \quad \text{with} \quad \mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k)$$

- $(\mathbf{L}_k)_{1 \leq k \leq K}$ are convolution operators
- $(\mathfrak{D}_{\Lambda_k})_{1 \leq k \leq K}$ are a combination of rectified linear units (ReLU) and max-pool/up-conv for upsampling/downsampling operations

Remark: Developed for image segmentation [Ronneberger et al., 2015]



NN example: U-Net

Definition: Unet is an encoder-decoder architecture defined by

$$f_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K} \quad \text{with} \quad \mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k)$$

- $(\mathbf{L}_k)_{1 \leq k \leq K}$ are convolution operators
- $(\mathfrak{D}_{\Lambda_k})_{1 \leq k \leq K}$ are a combination of rectified linear units (ReLU) and max-pool/up-conv for upsampling/downsampling operations

Remark: Unet architectures often use **skip-connections**:

- Considering a layer k where the variable needs to be “skipped”:

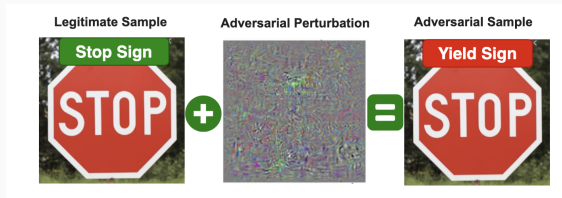
$$\mathfrak{T}_{\Theta_k}(\mathbf{x}) = \begin{pmatrix} \mathbf{x} \\ \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \mathbf{x} + \mathbf{b}_k) \end{pmatrix}$$

- **Skip-connections and inertia/momentum** share the same idea of reusing information from a past layer/iteration.

Robustness

How to control robustness of NNs?

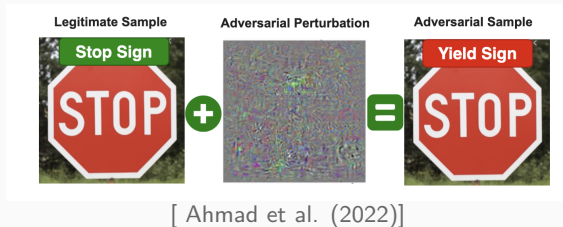
Idea: Use Lipschitz constant [Hoffman et al. (2019)] [Jakubovitz and Giryes (2018)] [Pesquet et al. (2021)]



[Ahmad et al. (2022)]

How to control robustness of NNs?

Idea: Use Lipschitz constant [Hoffman et al. (2019)] [Jakubovitz and Giryes (2018)] [Pesquet et al. (2021)]



Definition: Let an input $x \in \mathbb{R}^{N_0}$ and a perturbation $\epsilon \in \mathbb{R}^{N_0}$.

The error on the output can be bounded as: $\|f_{\Theta}^K(x + \epsilon) - f_{\Theta}^K(x)\| \leq \omega \|\epsilon\|$

- Remarks:**
- $\omega > 0$ is (by definition) a Lipschitz constant of f_{Θ}^K
 - ω : certificate of the network robustness
 - Important and complementary to the performance in terms of estimation quality

Robustness: Control of the Lipschitz constant

Definition: Let an input $\mathbf{x} \in \mathbb{R}^{N_0}$ and a perturbation $\epsilon \in \mathbb{R}^{N_0}$.

The error on the output can be bounded as: $\|\mathbf{f}_{\Theta}^K(\mathbf{x} + \epsilon) - \mathbf{f}_{\Theta}^K(\mathbf{x})\| \leq \omega \|\epsilon\|$

- Layer-by-layer certificate:**

Let $\mathbf{f}_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K}$, with $\mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k)$.

If $\mathfrak{D}_{\Lambda_k}$ is nonexpansive, then a Lipschitz constant for $\mathbf{f}_{\Theta}^K$ is $\omega = \prod_{k=1}^K \|\mathbf{L}_k\|_S$

- Global certificate:**

If $\mathbf{f}_{\Theta}^K$ is differentiable, its Lipschitz constant is given by $\omega = \sup_{\mathbf{x} \in \mathbb{R}^{N_0}} \|\nabla \mathbf{f}_{\Theta}^K(\mathbf{x})\|_S$ where

$\nabla \mathbf{f}_{\Theta}^K(\mathbf{x}) = \left(\frac{\partial_i \mathbf{f}_{\Theta}^K(\mathbf{x})}{\partial \mathbf{x}} \right)_{1 \leq i, j \leq N_0}$ is the Jacobian of $\mathbf{f}_{\Theta}^K$ at $\mathbf{x} \in \mathbb{R}^{N_0}$.

Layer-by-layer certificate

- **Layer-by-layer certificate:**

Let $f_{\Theta}^K = \mathfrak{T}_{\Theta_1} \circ \dots \circ \mathfrak{T}_{\Theta_K}$, with $\mathfrak{T}_{\Theta_k} = \mathfrak{D}_{\Lambda_k}(\mathbf{L}_k \cdot + \mathbf{b}_k)$. If $\mathfrak{D}_{\Lambda_k}$ is nonexpansive, then a Lipschitz

constant for f_{Θ}^K is $\omega = \prod_{k=1}^K \|\mathbf{L}_k\|_S$

Remarks:

- Bound not tight
- Bounds can often be overly pessimistic in practice, thus providing limited insights into the robustness of the network [Combettes, Pesquet (2020)][Neacsu et al. (2024)].

Global certificate

- **Global certificate:**

If f_{Θ}^K is differentiable, its Lipschitz constant is given by $\omega = \sup_{x \in \mathbb{R}^{N_0}} \|\nabla f_{\Theta}^K(x)\|_S$

Remarks:

- Impossible to compute over all images in $\mathbb{R}^N \rightsquigarrow$ Known to be an NP-hard problem
- Restrict the computation to a set of images $(x_i)_{i \in \mathbb{I}}$ [Pesquet et al. 2021], i.e.

$$\omega \approx \max_{(x_i)_{i \in \mathbb{I}}} \|\nabla f_{\Theta}^K(x_i)\|_S$$

- $\rightsquigarrow$ Underestimation of the Lipschitz constant.
- $\rightsquigarrow$ Computed using power iterations coupled with automatic differentiation.
- $\rightsquigarrow$ Used for constraining the value of ω [Pesquet et al. 2021] (*see section PnP*)
- $\rightsquigarrow$ Used to compare robustness of different architectures [Le et al. 2023]

Some methods to control the Lipschitz constant of NN layers

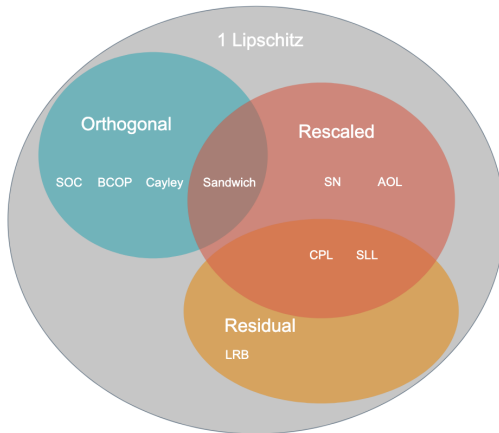


Fig. 2.1.: Venn diagram illustrating the categorization of Lipschitz layers, including scaled layers, orthogonal layers, and residual layers.

Links with proximity operators

Activation functions and proximity operators

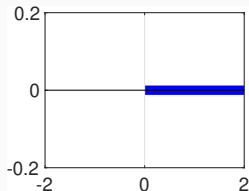
Remark: Most activation functions used in imaging are proximity operators

[Combettes & Pesquet, 2020]

Examples: ReLu

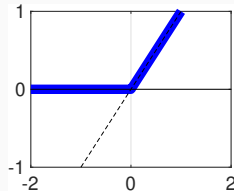
Potential

$$\varphi(x) = \iota_{[0,+\infty)}$$



Activation/ proximity operator

$$\mathfrak{D}_{\Lambda}(x) = \text{prox}_{\varphi}(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



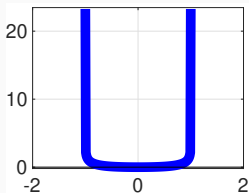
Activation functions and proximity operators

Remark: Most activation functions used in imaging are proximity operators

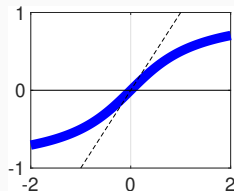
[Combettes & Pesquet, 2020]

Examples: Arctangent

$$\varphi(x) = \begin{cases} \text{Potential} \\ -\frac{2}{\pi} \ln \left(\cos \left(\frac{\pi x}{2} \right) \right) - \frac{x^2}{2} & \text{if } |x| < 1 \\ +\infty & \text{otherwise} \end{cases}$$



$$\text{Activation/ proximity operator} \\ \mathfrak{D}_{\Lambda}(x) = \text{prox}_{\varphi}(x) = \frac{2 \arctan(x)}{\pi}$$



Activation functions and proximity operators

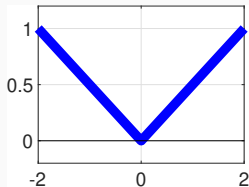
Remark: Most activation functions used in imaging are proximity operators

[Combettes & Pesquet, 2020]

Examples: **Soft thresholding**

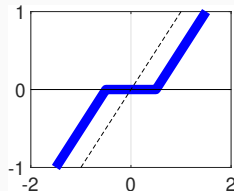
Potential

$$\varphi(x) = \lambda|x|$$



Activation/ proximity operator

$$\mathcal{D}_\Lambda(x) = \text{prox}_\varphi(x) = \text{sign}(x) \max\{|x| - \lambda, 0\}$$



Link between activation functions and proximity operators

Proposition [Combettes & Pesquet, 2020] [Gharbi *et al.*, 2023]

Let $\varrho: \mathcal{H} \rightarrow \mathcal{H}$ such that, there exists a **differentiable** function $\psi: \mathcal{H} \rightarrow \mathbb{R}$ where

$$\varrho = \nabla \psi .$$

Let $\beta \in (0, +\infty)$, $\mu \in (-1, +\infty)$ be such that $\beta = 1/(1 + \mu)$ and φ is μ -**convex**.

Then

$$\varrho = \text{prox}_{\varphi} \Leftrightarrow \psi \text{ is convex with a } \beta\text{-Lipschitz continuous gradient}$$

Reminder:

φ is μ -convex if $\varphi - \mu \frac{\|\cdot\|^2}{2}$ is convex

Activation functions, potentials and penalizations: Some examples

Name	Prox/Activation $x \in \mathbb{R} \mapsto \varrho(x) = \text{prox}_{\varphi}(x) = \psi'(x)$	Potential $x \in \mathbb{R} \mapsto \varphi(x)$	Penalization $x \in \mathbb{R} \mapsto \psi(x)$
ReLU	$\begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$ 	$\ell_{[0,+\infty)}$ 	$\begin{cases} \frac{x^2}{2} & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$ 
Parametric ReLU	$\begin{cases} x & \text{if } x > 0 \\ \lambda x & \text{otherwise} \end{cases}$ 	$\begin{cases} 0 & \text{if } x > 0 \\ \frac{(1/\lambda - 1)x^2}{2} & \text{otherwise} \end{cases}$ 	$\begin{cases} \frac{x^2}{2} & \text{if } x > 0 \\ \lambda \frac{x^2}{2} & \text{otherwise} \end{cases}$ 
Bent identity	$\frac{x + \sqrt{x^2 + 1} - 1}{2}$ 	$\begin{cases} \frac{x}{2} - \frac{\ln(x + 1/2)}{4} & \text{if } x > -1/2 \\ +\infty & \text{otherwise} \end{cases}$ 	$\frac{x^2 + x\sqrt{x^2 + 1} - 2x + \text{arcsinh}(x)}{4}$ 
Inverse square root	$\frac{x}{\sqrt{1+x^2}}$ 	$\begin{cases} -\frac{x^2}{2} - \sqrt{1-x^2} & \text{if } x \leq 1 \\ +\infty & \text{otherwise} \end{cases}$ 	$\sqrt{1+x^2}$ 
Inverse square root linear unit	$\begin{cases} x & \text{if } x \geq 0 \\ \frac{x}{\sqrt{1+x^2}} & \text{otherwise} \end{cases}$ 	$\begin{cases} 0 & \text{if } x \geq 0 \\ 1 - \frac{x^2}{2} - \sqrt{1-x^2} & \text{if } -1 \leq x < 0 \\ +\infty & \text{otherwise} \end{cases}$ 	$\begin{cases} \frac{x^2}{2} & \text{if } x \geq 0 \\ \sqrt{1+x^2} - 1 & \text{otherwise} \end{cases}$ 
Arctangent	$\frac{2 \arctan(x)}{\pi}$ 	$\begin{cases} -\frac{2}{\pi} \ln\left(\cos\left(\frac{\pi x}{2}\right)\right) - \frac{x^2}{2} & \text{if } x < 1 \\ +\infty & \text{otherwise} \end{cases}$ 	$\frac{2 \arctan(x)x - \ln(x^2 + 1)}{\pi}$ 
Unimodal sigmoid	$\frac{1}{1 + \exp(-x)} - \frac{1}{2}$ 	$\begin{cases} (\frac{1}{2} + x) \ln(\frac{1}{2} + x) + (\frac{1}{2} - x) \ln(\frac{1}{2} - x) & \text{if } x < 1/2 \\ -\frac{1}{2}(x^2 + \frac{1}{4}) & \text{if } x = 1/2 \\ -1/4 & \text{if } x = 1/2 \\ +\infty & \text{otherwise} \end{cases}$ 	$\frac{x}{2} + \ln(1 + \exp(-x))$ 

Activation functions & proximity operators: Weakly convex case

Proposition [Hurault *et al.*, 2022]

Let $\beta \in [0, 1]$ and let $\psi: \mathcal{H} \rightarrow \mathbb{R}$ be a differentiable function with a β -Lipschitz gradient. Then, there exists a $\beta/(\beta + 1)$ -weakly convex function $\varphi: \mathcal{H} \rightarrow (-\infty, +\infty]$, such that

$$\text{Id} - \nabla \psi = \text{prox}_{\varphi}$$

Remark: If φ is convex, proper and l.s.c., then ψ is the Moreau envelope of φ defined as

$$(\forall \tilde{x} \in \mathcal{H}) \quad \psi(\tilde{x}) = \inf_{x \in \mathcal{H}} \varphi(x) + \frac{1}{2} \|x - \tilde{x}\|^2.$$

Conclusion

Conclusion

